

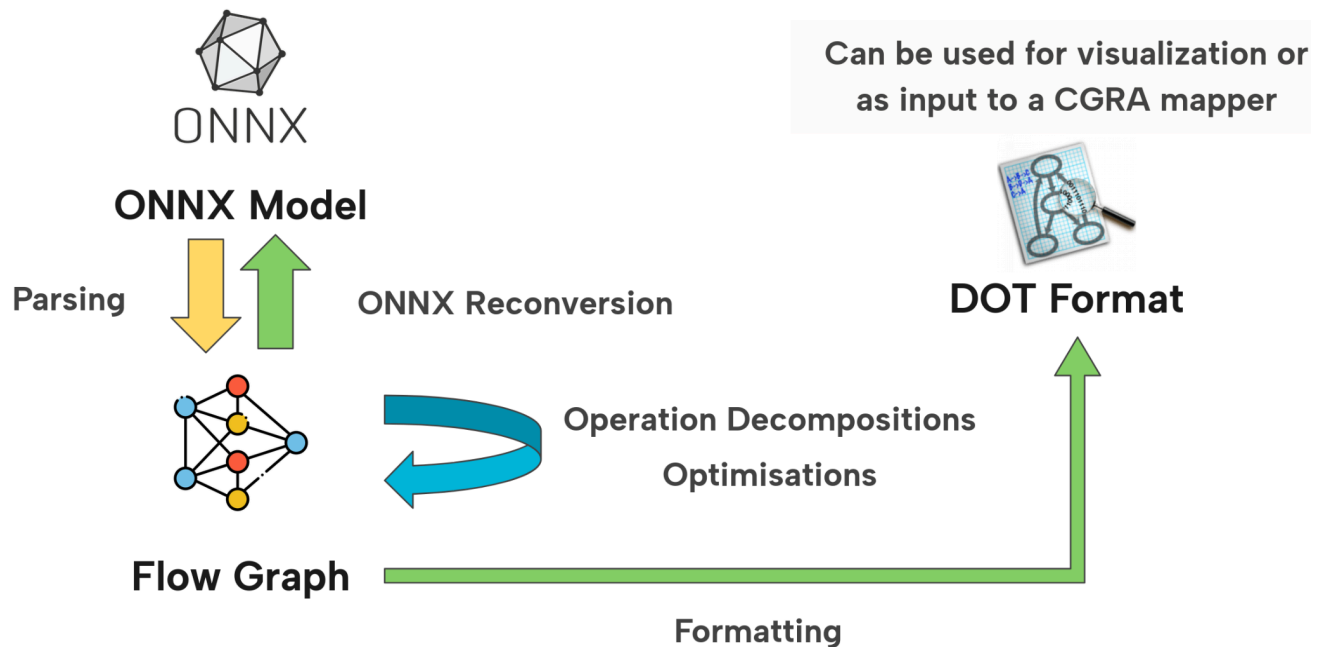
ONNX-Flow

Context: The rapid growth of Edge AI has created a demand for real-time, low-power artificial intelligence on embedded devices. Traditional hardware, such as CPUs and GPUs, is often too bulky or energy-inefficient for these constraints. Coarse-Grained Reconfigurable Arrays (CGRAs) offer a promising alternative by balancing hardware specialization with programmability, providing a better performance-energy balance. However, the deployment of applications on CGRAs is currently hindered by a lack of practical toolchains, specifically for handling standard AI models.

Motivation: While ONNX serves as a portable standard for representing AI models as graphs, there is a "semantic gap" between these high-level representations and the low-level requirements of CGRAs. CGRAs typically optimize for inner loops of kernels, whereas ONNX models use high-level operators, like MatMul, although the format also supports lower-level operations. ONNX-Flow is designed to bridge this gap by offloading ONNX models to CGRAs through automatic decomposition, transforming high-level operations into structures that hardware accelerators can process efficiently.

Implementation & Features: ONNX-Flow acts as a bridge that transforms standard ONNX models into decomposed Data Flow Graphs (DFGs).

- **Inputs & Outputs:** The tool consumes standard ONNX models and generates a DFG (often in DOT format for visualization or as input for CGRA mappers). It can also output ONNX models (reconversion with the applied transformations) and the corresponding JSON.
- **Core Functions:**
 - **Operation Decomposition:** High-level operators (e.g., MatMul, Add, Div) are lowered into explicit loops and low-level scalar operations such as Gather, Scatter, and fundamental arithmetic.
 - **Shape Inference:** It automatically infers the shapes of intermediate tensors using operator semantics.
 - **Optimizations:** The tool detects and merges compatible operations into fused loop nests (Loop Fusion) and offers loop coalescing.



Test Models & Implementation: To validate the framework, the team utilized a mix of unit tests and standard Edge AI applications, ranging from simple arithmetic to full neural networks.

- **Test Models Considered:**
 - **ad01_fp32 (TinyML):** An Anomaly Detection model representing standard tabular data processing.
 - **kws_ref_model_fp32 (TinyML):** A Keyword Spotting model (audio processing), chosen to test the decomposition of complex **Conv** (convolution) layers.
 - **SC2_X & matmul_test:** Smaller unit-test models used to verify specific matrix multiplication scenarios.
- **Transformation Process:** The tool performs a massive structural expansion to bridge the semantic gap between software and hardware. It totally eliminates high-level "black box" operators, replacing them with explicit control flow and memory addressing nodes: Complex operators like **Gemm** (General Matrix Multiply), **Conv** (Convolution), and **MatMul** are reduced to 0 occurrences in the generated graph. These are replaced by (decomposed in) thousands of low-level primitives. For example, in the KWS model, 9 **Conv** nodes were replaced by a combination of **Loop** (30), **Mul** (3913), **Add** (3878), and memory ops like **Gather/Unsqueez** (over 2000 combined).

Measured Results: The primary validation metric was functional equivalence, ensuring that the decomposed hardware-ready graphs produced identical numerical results to the original high-level software models.

- **Functional Equivalence (100% Pass Rate):**
 - The platform successfully verified that the output tensors of the decomposed models matched the original models exactly.
 - For example, the `ad01` model inputs yielded identical vectors (e.g., starting with `-37.85...`) in both the standard and reconverted versions, confirming that the complex loop transformations preserved the mathematical integrity of the AI model.
- **Structural Complexity (Node Count):**
 - The logs quantify the "semantic gap" as a massive increase in graph complexity.
 - Anomaly Detection (`ad01`): Increased from **19** original nodes to **666** decomposed nodes.
 - Keyword Spotting (`kws`): Exploded from **28** original nodes to **12,448** decomposed nodes, illustrating the intense level of detail required to define convolution loops for hardware.
- **Simulation Performance:**
 - While not the target metric for hardware deployment, the logs show that running these decomposed models in a standard software interpreter (CPU) is significantly slower (e.g., ~26s vs 6ms for KWS), confirming that these structures are intended for custom hardware (CGRAs) rather than general-purpose CPU execution or need further optimizations that the decomposition unlocks.